

WMBus PHY Demo Guide

ESMT WISE SDK V2 - WISEDemoApp on ER8130A / ER8131A EVK

1. Overview

This guide describes how to run the Wireless M-Bus (WMBus) PHY demo on two ESMT EVKs. One board acts as a **meter** and the other as a **collector** ("other" device). After power-on the boards exchange WMBus T2 frames automatically; the UART shell can pause the automatic demo and run a packet error rate (PER) test in any supported PHY mode.

1.1 Hardware and firmware setup

Item	EVK A	EVK B
Boot loader (pre-flashed)	demo_boot_loader	demo_boot_loader
Application (pre-loaded @ 0x6000)	demo_wmbus_phy_meter	demo_wmbus_phy_other
WMBus role	Meter	Other (collector)
WMBus mode at boot	T2 (bidirectional)	T2 (bidirectional)
Device ID (BCD)	0xB0000001	0xA0000001
Console	UART0, 8N1, terminal program	UART0, 8N1, terminal program

Both applications are built from the same source (app/WISEDemoApp/src/demo_wmbus/); the role is selected at build time by the WMBUS_DEMO_PHY_METER or WMBUS_DEMO_PHY_OTHER define. Connect a terminal to each EVK so that both consoles are visible at the same time.

1.2 Flash layout

Flash offset	Content	Notes
0x0000	demo_boot_loader	Runs first on every reset
0x6000	Application image	Fixed boot address (APP_BOOT_ADDR); updated via kermi

2. Boot loader flow

On every reset `demo_boot_loader` initialises the core, prints its banner and then gives the user a short window to enter the loader console. If nothing is entered, it boots the application at flash offset 0x6000.

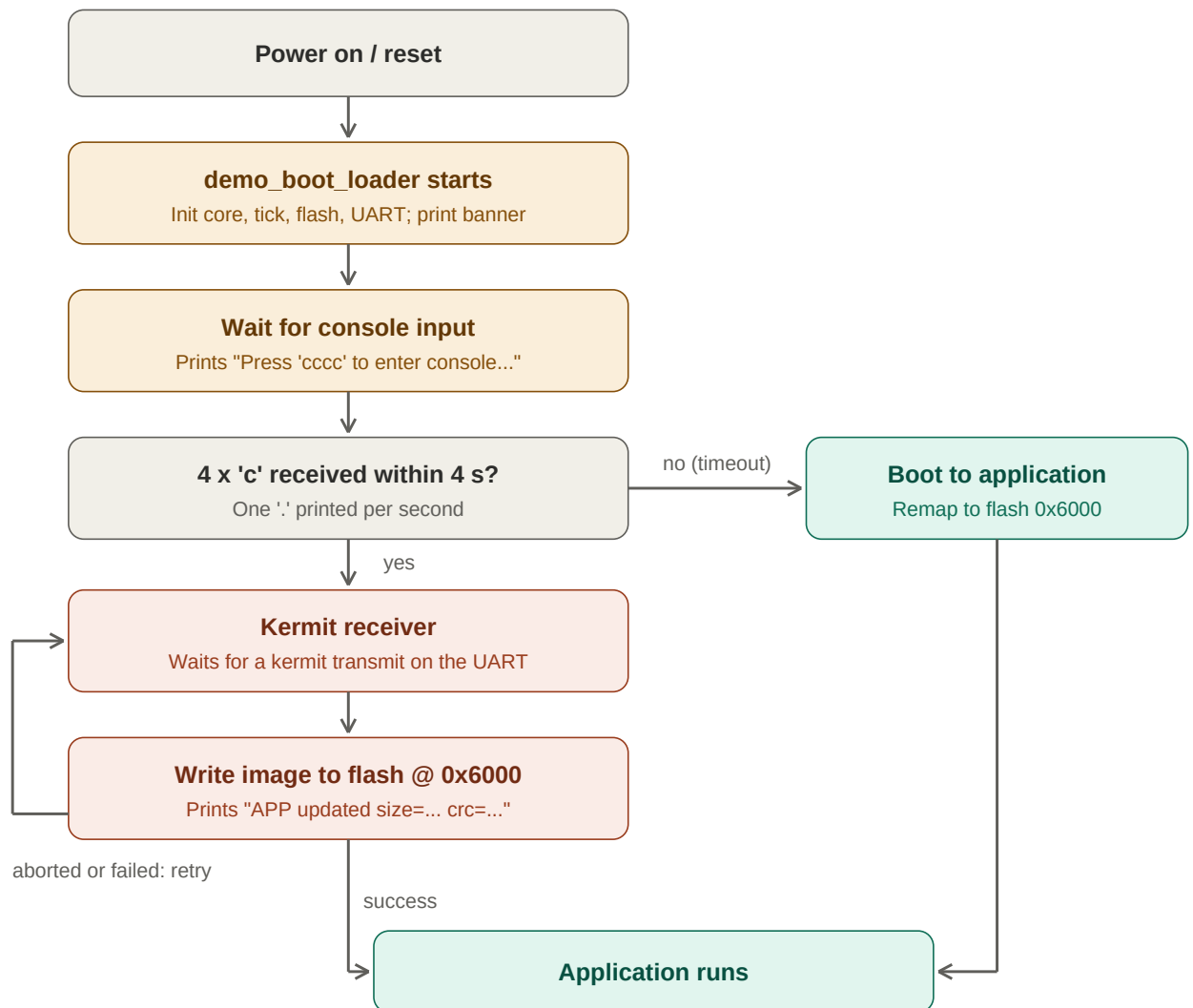


Figure 1 - `demo_boot_loader` decision flow

2.1 Normal boot

- The loader prints `Press 'cccc' to enter console...` and then one dot per second.
- If four c characters are not received within the 4-second window, it prints `booting to APP @00006000`, disables the UART and peripherals, and remaps to the application.
- The application banner (`ESMT WISE Demo Application: DEMO W-Mbus`) follows immediately.

2.2 Firmware update via kermit

To load a new application image, type `cccc` in the terminal right after the loader banner appears (within the 4-second window). The loader then starts a kermit receiver and waits for the host to send the image:

- Console shows `Start kermit receiver`.
- In the terminal program, start a **kermit transmit** (send file) of the application binary (for example `demo_wmbus_phy_meter.bin`).

- The image is written to flash at 0x6000. On success the loader prints APP updated size=xxxxxxx crc=xxxx and boots the new application.
- If the transfer is aborted or fails, the loader prints aborted and restarts the kermit receiver; it keeps waiting until a transfer succeeds. Reset the board to leave this state.

Note: the current loader source (`_loader_wait_user_input()`) requires four 'c' characters within four 1-second checks. If the intended behaviour is three characters within 3 seconds, adjust `checkCount` and the `hitCounter` threshold in `demo_boot_loader.c` and update this guide.

3. Application start

Both EVKs run the same loader; only the application image differs. Each application initialises the WMBus radio in T2 mode with its build-time role, sets the demo AES key, prints a short command guide and starts the automatic demo.

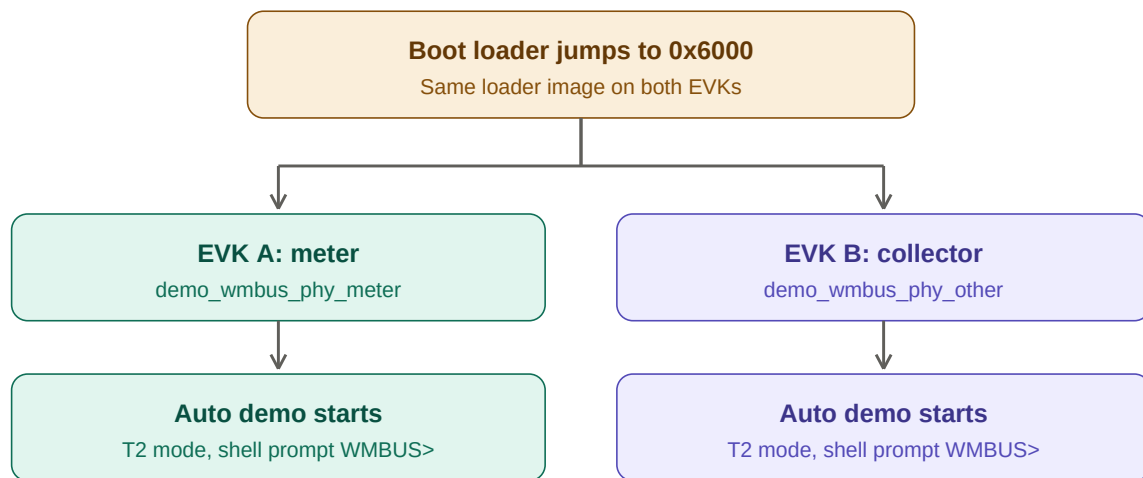


Figure 2 - Application start on the two EVKs

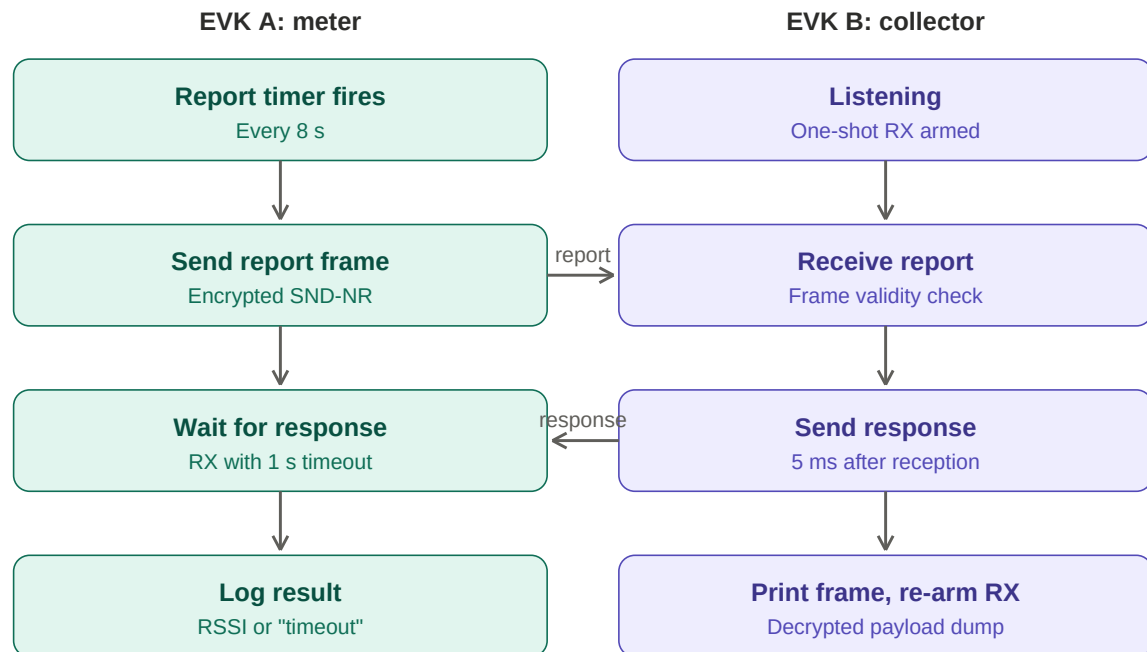
Console output after boot (meter shown; the collector prints the same block with its own description):

```

Start WMBus T2-METER
Auto demo: meter sends a report every 8000 ms and waits for the collector response
Commands:
demo start|stop          switch the auto demo on/off
per [mode] tx [count] [interval_ms] PER test sender
per [mode] rx [count]    PER test receiver (prints PER)
                        [mode] 0=S2, 1=T2, 2=C2, 3=R2 (Ctrl+C aborts a running test)
help                    list all commands
WMBUS>
  
```

4. Automatic demo

No user action is required. The meter reports every 8 seconds and the collector answers each report. Both consoles log the exchange with a timestamp and RSSI.



Cycle repeats every 8 s until "demo stop" is entered

Figure 3 - Automatic meter / collector exchange

4.1 Meter (EVK A)

- A periodic scheduler entry fires every 8 s and the meter task builds an encrypted (AES mode 5) SND-NR frame carrying a volume and a volume-flow value.
- After the TX-done event the meter opens a one-shot RX window and waits up to 1 s for the collector response.
- Console: `WMbus meter report N, then received response rssi=-xx or wait response timeout.`

4.2 Collector (EVK B)

- The collector task keeps a one-shot RX window open. When a valid frame arrives it replies with a null frame 5 ms after the reception timestamp, then decrypts and prints the received frame.
- Console: `received meter report rssi=-xx` followed by the DLL header, application header and payload bytes.
- RX is re-armed after every frame, valid or not.

4.3 Switching the auto demo off and on

Command	Effect
<code>demo stop</code>	Stops the periodic report (meter) or the RX loop (collector). The task is kept idle.
<code>demo start</code>	Resumes the automatic demo. Entering it while already running just prints a message.

5. PER test from the shell

The `per` command measures the packet error rate between the two boards in a selectable PHY mode. Either EVK can be the sender or the receiver; the example below uses the meter as sender. Both sides must use the same mode index.

5.1 Command syntax

Command	Parameters
<code>per [mode] tx [count] [interval_ms]</code>	Send <i>count</i> numbered frames, one every <i>interval_ms</i> milliseconds.
<code>per [mode] rx [count]</code>	Listen until <i>count</i> frames are received, then print the PER report.
<code>[mode]</code>	0 = S2, 1 = T2, 2 = C2, 3 = R2 (same index as <code>wmbus init</code> in <code>wise_core_trunk</code>).

5.2 Procedure

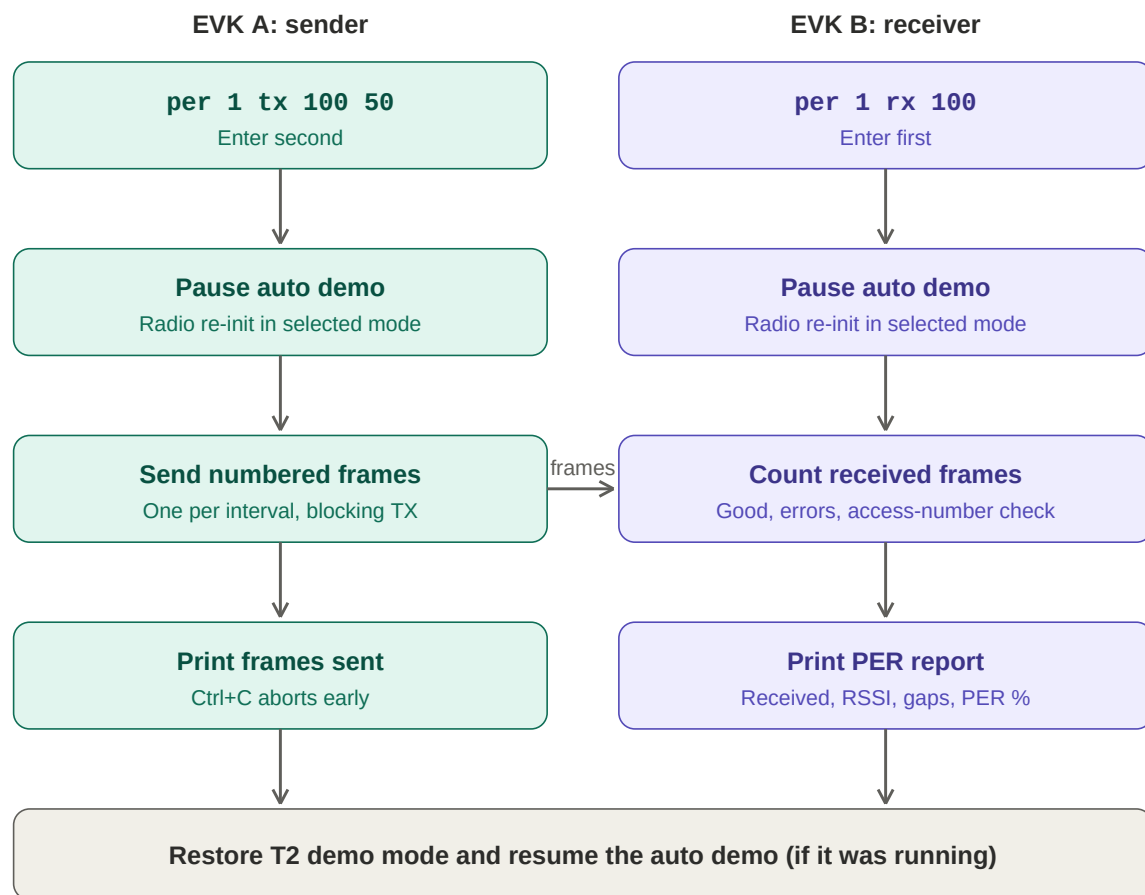


Figure 4 - PER test sequence (example: T2 mode, 100 frames, 50 ms interval)

- **Receiver first:** on EVK B enter `per 1 rx 100`. The auto demo pauses, the radio is re-initialised in T2 and the board prints `PER rx: expecting 100 frames, Ctrl+C to stop`.
- **Then the sender:** on EVK A enter `per 1 tx 100 50`. It sends 100 short unencrypted SND-NR frames (DLL + STL header only, 15 bytes, L=14) 50 ms apart; the STL access-number field carries the sequence 0, 1, 2, ... One > is printed per frame.
- The receiver prints one character per valid frame: < when the access number is the previous one + 1, = when it repeats the previous one, X when it jumped elsewhere.

- The receiver stops when 100 frames have arrived, after 5 s without a frame (once the first one was seen), or on Ctrl+C.
- Both boards restore the T2 demo configuration afterwards and resume the auto demo if it was running before the test.

5.3 Report

```
PER test result:
  97/100 frames received
  rx error frames: 2
  access number: 94 in order, 0 repeated, 2 gaps
  average rssi: -71
  PER: 3.00%
```

Field	Meaning
frames received	Valid frames counted / frames expected (the rx count argument).
rx error frames	Radio RX error events or frames that failed the validity check (detected corruption). Not counted as received.
access number	Classification of every valid frame after the first against the previous frame: <i>in order</i> = previous + 1, <i>repeated</i> = same value, <i>gaps</i> = anything else (frames lost in the air). All three count as received.
average rssi	Mean RSSI over the valid frames, in dBm.
PER	(expected - received) / expected, in percent.

5.3.1 Using a signal generator as the source

The receiver does not require the sender to be an EVK. A signal generator replaying one fixed WMBus frame can be used instead: every replayed frame still counts as received, so PER is calculated the same way. Because the access number never changes, the report shows all frames as *repeated* and adds a note:

```
PER test result:
  100/100 frames received
  rx error frames: 0
  access number: 0 in order, 99 repeated, 0 gaps
  (access number never changed: fixed-pattern transmitter, gap count not meaningful)
  average rssi: -68
  PER: 0.00%
```

With a fixed-pattern source, set the receiver's *count* to the number of frames the generator will send; lost frames then show up as a PER above 0 % (and as an idle timeout if the last ones are missing), since there is no sequence information to detect them.

5.4 Notes

- Changing the PHY mode requires a full radio de-init / re-init; this is done automatically by the command.
- The sender's role stays the build-time role (meter on EVK A, other on EVK B), so the direction of the link is meter-to-other when EVK A sends and other-to-meter when EVK B sends.
- The access number is 8-bit and wraps at 256; the in-order check handles the wrap-around.
- The PER frame is built by `wmbus_setup_null_frame()`: L=14, C=SND-NR, manufacturer ESM, device type 0x00 (other), CI=0x7A, encryption mode 0.
- The 5-second receiver idle timeout is `PER_RX_IDLE_TIMEOUT_MS` in `demo_wmbus.c`; raise it for very long TX intervals.

5.5 PER test pattern (byte arrays)

This section shows how the PER frame of the **meter build in T2 mode with access number 0** is packed: the raw packet, the same packet in frame format A, and the 3-of-6 encoded result. The arrays can be used to program a signal generator or to check a captured frame.

Step 1 - Raw packet (15 bytes)

DLL header (10 bytes) followed by the short transport layer (STL) header (5 bytes). There is no payload. The L-field counts every byte after itself, so L = 14.

```
const uint8_t perRawPacket[15] = {
    0x0E,          /* L = 14 */
    0x44,          /* C: primary, SND-NR */
    0x6D, 0x16,    /* M-field "ESM" = 0x166D, little-endian */
    0x01, 0x00, 0x00, 0xB0, /* ID = 0xB0000001 (BCD), little-endian */
    0x02,          /* version */
    0x00,          /* device type = OTHER */
    0x7A,          /* CI = EN13757-3 application, short TL */
    0x00,          /* access number (sequence) */
    0x00,          /* status */
    0x00, 0xA0,    /* conf word: mode 0, sync=1, accessibility=limited */
};
```

Step 2 - Frame format A (19 bytes)

Frame format A splits the packet into blocks and appends a CRC to each one. The first block is always the 10-byte DLL header; the following blocks hold up to 16 bytes each. This packet has one more block with the 5 STL bytes. The CRC is CRC-16 EN 13757 (polynomial 0x3D65, initial value 0, result inverted) and is stored high byte first.

Block	Data bytes	CRC	Size
1 (DLL header)	0E 44 6D 16 01 00 00 B0 02 00	10 C7	10 + 2
2 (STL header)	7A 00 00 00 A0	35 A8	5 + 2

```
const uint8_t perFrameA[19] = {
    0x0E, 0x44, 0x6D, 0x16, 0x01, 0x00, 0x00, 0xB0, 0x02, 0x00, /* block 1 */
    0x10, 0xC7,                                                  /* CRC of block 1 */
    0x7A, 0x00, 0x00, 0x00, 0xA0,                                /* block 2 */
    0x35, 0xA8,                                                  /* CRC of block 2 */
};
```

Step 3 - 3-of-6 encoding (29 bytes)

T-mode meter-to-other frames are 3-of-6 encoded: every 4-bit nibble becomes a 6-bit symbol that contains exactly three ones. Each byte is encoded high nibble first, so one byte becomes 12 bits.

Nibble	Symbol	Nibble	Symbol	Nibble	Symbol	Nibble	Symbol
0	010110	4	011100	8	101100	C	110100
1	001101	5	011001	9	100101	D	110001
2	001110	6	011010	A	100110	E	110010
3	001011	7	010011	B	100011	F	101001

- Example: the first byte 0x0E has nibbles 0 and E, giving 010110 110010.
- 19 bytes give 19 x 12 = 228 bits. Because the byte count is odd, a 4-bit postamble 0101 is appended, for 232 bits = 29 bytes.
- The array below lists the encoded bit stream packed MSB first, i.e. in the order the bits go on air.

```
const uint8_t perEncoded3of6[29] = {
    0x5B, 0x27, 0x1C, 0x6B, 0x13, 0x5A, 0x58, 0xD5,
    0x96, 0x59, 0x68, 0xD6, 0x58, 0xE5, 0x96, 0x35,
    0x6D, 0x13, 0x4E, 0x65, 0x96, 0x59, 0x65, 0x96,
    0x99, 0x62, 0xD9, 0x9A, 0xC5,
};
```

- The preamble and sync word are not part of this array. On air they come first: the T-mode preamble followed by the sync word 0x543D, then these 29 bytes.
- With `per . . . tx` the access number (byte 11 of the raw packet) increments per frame, which changes the CRC of block 2 and its encoding.
- The other build (ID 0xA0000001) and other PHY modes differ: S mode and T2 other-to-meter use Manchester encoding, C mode uses NRZ.

6. Quick reference

Where	Command / action	Result
Loader (first 4 s)	<code>cccc</code>	Enter kermit receiver for firmware update
Loader	<code>(no input)</code>	Boot application at 0x6000
Application	<code>demo stop</code>	Pause the automatic demo
Application	<code>demo start</code>	Resume the automatic demo
Application	<code>per [mode] rx [count]</code>	Start PER receiver, prints PER report
Application	<code>per [mode] tx [count] [interval_ms]</code>	Start PER sender
Application	<code>Ctrl+C</code>	Abort a running PER test
Application	<code>help</code>	List all shell commands
Application	<code>reset</code>	Chip reset (returns to the loader)